DBMS PROJECT REPORT

Campus Event Portal

A web-based system for campus events, registrations, resources and attendance.

PROJECT OVERVIEW

Discover events. Manage registrations. Coordinate campus resources.
Record attendance with QR check-in.

HTML / CSS / JavaScript | Node.js + Express | MySQL | PDFKit + QR Code

Prepared from the project source and database schema

2026

PROJECT OVERVIEW

Introduction, problem and objectives

02. INTRODUCTION

One portal for campus life

Campus Event Portal is a full-stack web application that helps students discover campus events and manage participation. Organizers can publish events and reserve rooms or shared equipment. The system supports capacity-aware registration, waitlisting, QR check-in, attendance records and participation certificates.

03. PROBLEM STATEMENT

The challenge it addresses

Event coordination often relies on separate forms, spreadsheets and manual room bookings. This makes it difficult to track participant limits, handle cancellations, avoid overlapping resource reservations and keep attendance records accurate. The portal centralizes these workflows and uses database rules and transactions to keep records consistent.

04. OBJECTIVES

What the project aims to do

- Provide a searchable, filterable listing of campus events.
- Register students and manage full-event waitlists in FIFO order.
- Prevent conflicts when booking rooms and shared equipment.
- Record eligible attendees through event-specific QR check-in.
- Support attendance viewing and certificate downloads.

05. TECHNOLOGY USED

Application stack

Frontend

HTML, CSS and vanilla JavaScript
Responsive browser interface

Backend

Node.js and Express
REST-style API endpoints

Database & tools

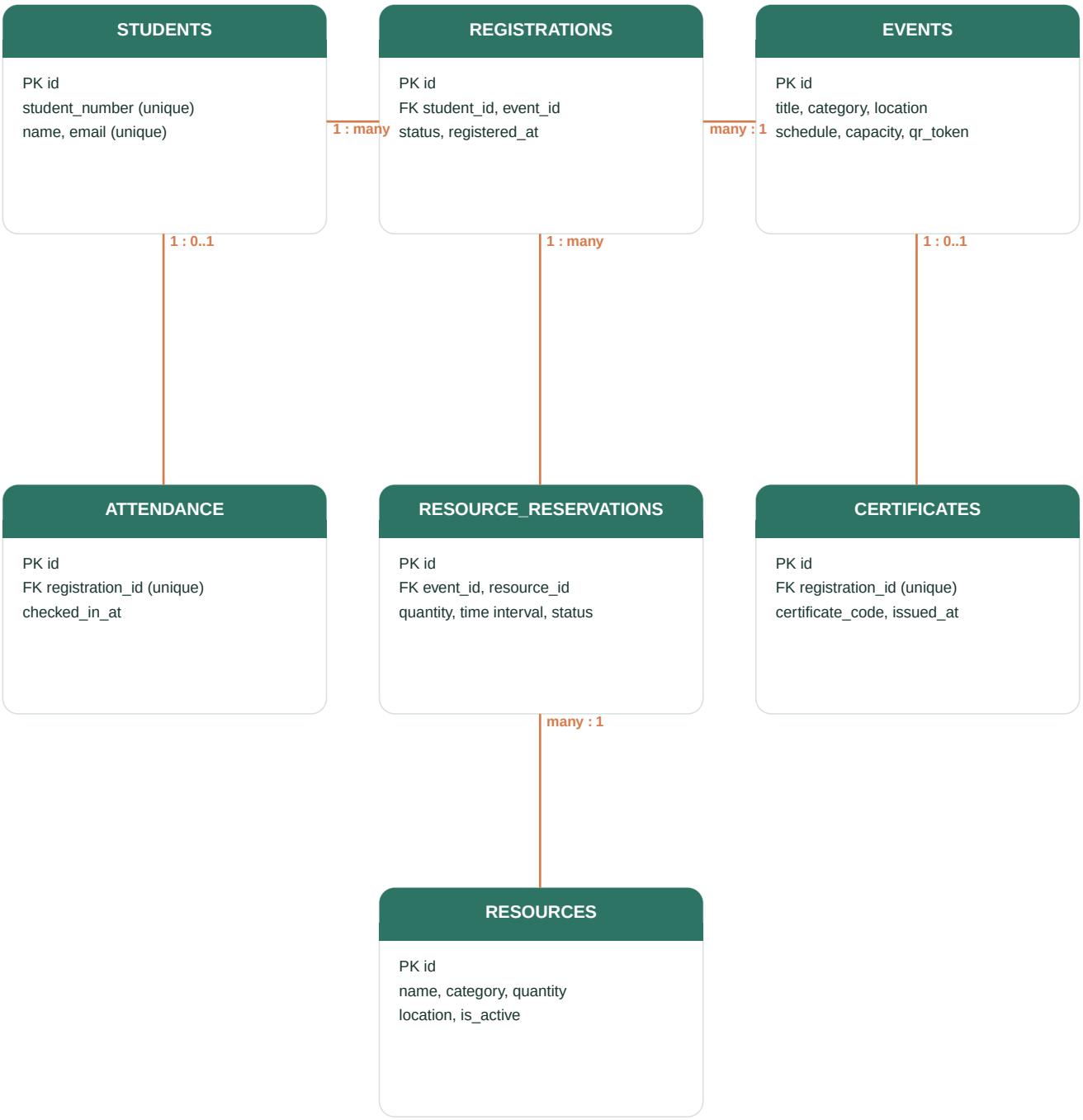
MySQL, mysql2, QRCode and PDFKit

Runtime requirement: Node.js 20+ and MySQL 8.0+.

DATA MODEL

ER diagram and relationships

The many-to-many relationship between students and events is resolved by registrations.



Cardinality note A student can have many registrations; each registration belongs to one student and one event. Attendance and certificates are optional, at most one per registration.

RELATIONAL DESIGN

Tables, keys and normalization

All tables use InnoDB. Each table has an unsigned auto-increment primary key unless noted.

TABLE	IMPORTANT ATTRIBUTES	KEYS / RULES
students	student_number, name, email	PK id; unique student_number and email
events	title, category, location, starts_at, ends_at, capacity, qr_token	PK id; unique QR token; time and capacity checks
registrations	student_id, event_id, status, registered_at	PK id; FKs to students/events; unique student-event pair
resources	name, category, quantity, location, is_active	PK id; unique category/name; positive quantity
resource_reservations	event_id, resource_id, quantity, starts_at, ends_at, status	PK id; FKs to events/resources; unique event-resource pair
attendance	registration_id, checked_in_at	PK id; unique FK to registrations
certificates	registration_id, certificate_code, issued_at	PK id; unique registration and certificate code

DESIGN NOTES

Integrity and normalization

- 1NF: columns store individual values; repeated student and event details are not copied into registration rows.
- 2NF: registration attributes depend on the complete student-event association.
- 3NF-oriented structure: student, event and resource details are stored separately from relationship-specific data.
- Foreign keys, unique constraints and CHECK constraints enforce integrity and valid values.

DATABASE IMPLEMENTATION

SQL operations and DBMS concepts

CORE CONCEPTS

How the database supports the workflows

- Parameterized SQL queries separate values from SQL statements.
- Transactions make registration, cancellation and waitlist promotion atomic.
- SELECT ... FOR UPDATE locks event/resource rows during availability checks.
- Indexes support schedule lookups, waitlist order and resource-conflict checks.
- JOINs and aggregate subqueries combine records and count available seats.

EXAMPLE: ACTIVE REGISTRATION COUNT

Capacity check

```
SELECT COUNT(*) AS total
FROM registrations
WHERE event_id = ? AND status = 'registered';
```

EXAMPLE: FIFO WAITLIST PROMOTION

When an active registration is cancelled

```
SELECT id, student_id
FROM registrations
WHERE event_id = ? AND status = 'waitlisted'
ORDER BY registered_at ASC
LIMIT 1 FOR UPDATE;
```

The selected row is promoted within the same transaction that removes the cancelled registration.

SELECTED API ENDPOINTS

Application-to-database operations

GET /api/events | POST /api/events/:id/register | DELETE /api/events/:id/register
PUT /api/events/:id/resources | POST /api/attendance/check-in | GET /api/events/:id/certificate/:studentId

INTERFACE AND VERIFICATION

Screenshots and test cases

Application screenshot section

Screenshots are not embedded in this generated report. Capture the running app and replace these marked areas before submission.

SCREENSHOT

1. Event discovery

Event list, filters and summary cards.

SCREENSHOT

2. Event publishing

New event form and event details.

SCREENSHOT

3. Registrations & resources

Student registrations and resource list.

SCREENSHOT

4. QR check-in

Check-in form and confirmation result.

10. TESTING

Validation test results

TEST	EXPECTED RESULT	RESULT
Valid local date	Formatted for MySQL	PASS **PASS**
Impossible/malformed dates	Rejected with validation error	PASS **PASS**
Positive whole number	Accepts valid; rejects invalid values	PASS **PASS**
Required text / body	Trims text; rejects empty or invalid body	PASS **PASS**

Automated validation suite: 5 tests passed (npm test).

WRAP-UP

Conclusion and future scope

11. CONCLUSION

Project summary and learning

Campus Event Portal demonstrates how a relational database can support a practical event-management workflow. The project combines related entities with foreign keys, uniqueness and validation constraints, and uses transactions to maintain event capacity and resource availability. It also brings together frontend development, a Node.js API, MySQL queries and automated tests.

Key learning outcomes

- Designing relational tables and resolving many-to-many relationships.
- Applying constraints, indexes, parameterized queries and transactions.
- Connecting browser-based user flows to database-backed APIs.
- Testing validation behavior and documenting system requirements.

12. FUTURE SCOPE

Ideas for further development

Security and roles

Add secure student/organizer authentication, role-based authorization and audit logging.

Student experience

Add reminders, calendar integration, richer search and mobile-friendly QR scanning.

Analytics

Provide attendance summaries, event popularity metrics and resource-utilization reports.

Operations

Add event updates/cancellations, admin tools, monitoring and database backups.

Deployment note The demo uses a student selector rather than production authentication; add authorization before real-world use.